

ЛР10 (3 пары / 6 часов)

«Журнал действий (лог): записываем изменения задач в файл. Подготовка к поддержке»

Что уже есть

- CRUD + JSON + import/export/backup
- валидация
- поиск/фильтр/сортировка
- статистика + экспорт отчёта

Что делаем в ЛР10

Добавим “профессиональную” вещь: **логирование** — запись действий в файл.

Например:

- добавили задачу
- изменили статус
- удалили
- импортировали

Это нужно, чтобы:

- понимать, что происходило
 - искать ошибки
 - восстанавливать историю
-

0) Итог ЛР10 (что должно работать)

✓ Есть файл лога `logs/app_YYYY-MM-DD.log` (или `logs/app.log`)

✓ При действиях добавляются строки:

- дата/время

- действие
- короткие данные (Id, Title)

✓ Лог пишется при:

- Add
- Update (редактирование)
- ChangeStatus
- Delete
- Import/Export/Backup (минимум Import)

✓ Программа не падает, если папки logs нет — она создаётся

1) Теория (важная и понятная)

1.1. Что такое лог

Лог — это “дневник”, куда программа записывает важные события.

Пример строки лога:

```
2025-12-29 12:40:12 ADD id=3 title="Купить тетрадь"
```

1.2. Зачем лог нужен в реальной жизни

- если пользователи жалуются “пропали задачи” — вы смотрите лог
- если импорт сломал данные — лог поможет понять когда и какой файл импортировали
- это основа поддержки (support)

1.3. Где хранить логирование

Есть два варианта:

1. писать лог прямо в `Program.cs` (проще новичкам)
2. сделать отдельный класс `Logger` (правильно и удобно)

Мы сделаем **Logger**, но очень простой.

2) Практика. Пара 1: создаём Logger в Storage

Шаг 1. Создать класс AppLogger

В проекте `TaskTracker.Storage`:

1. Откройте папку `Services`
 2. Right click → Add → Class...
 3. Название: `AppLogger.cs`
 4. Add
-

Шаг 2. Вставить код AppLogger

Откройте `AppLogger.cs` и вставьте:

```
namespace TaskTracker.Storage.Services;

public class AppLogger
{
    private readonly string _logDir;

    public AppLogger(string logDir)
    {
        _logDir = logDir;
        Directory.CreateDirectory(_logDir);
    }

    private string GetLogFilePath()
    {
        // один файл на день
        var fileName = $"app_{DateTime.Now:yyyy-MM-dd}.log";
    }
}
```

```
        return Path.Combine(_logDir, fileName);
    }

    public void Info(string message)
    {
        var line = $"{DateTime.Now:yyyy-MM-dd HH:mm:ss} INFO {message}";
        WriteLine(line);
    }

    public void Error(string message)
    {
        var line = $"{DateTime.Now:yyyy-MM-dd HH:mm:ss} ERROR {message}";
        WriteLine(line);
    }

    private void WriteLine(string line)
    {
        var path = GetLogFilePath();
        File.AppendAllText(path, line + Environment.NewLine);
    }
}
```

Шаг 3. Сборка

Build → Build Solution → Build succeeded.

Шаг 4. Коммит №1

Сообщение:

- feat: add AppLogger service for daily logs

Commit All → Push.

3) Практика. Пара 2: подключаем Logger в App и логируем основные действия

Шаг 5. Открыть Program.cs

TaskTracker.App → Program.cs

Шаг 6. Создать logger рядом с storage

Найдите где у вас пути `dataFilePath`, `backupsFolder`, `exportsFolder`.

Добавьте путь для логов и создайте логгер:

```
var logsFolder = Path.Combine(AppContext.BaseDirectory, "logs");
var logger = new AppLogger(logsFolder);

logger.Info("Application started");
```

Проверьте, что сверху есть:

```
using TaskTracker.Storage.Services;
```

Шаг 7. Логируем Add

Найдите блок `input == "1"` (Добавить задачу).

Внутри `try` после успешного добавления вставьте:

```
logger.Info($"ADD id={task.Id} title=\"{task.Title}\"");
```

Шаг 8. Логируем Update (редактирование)

Найдите блок `input == "5"` (Редактировать).

После успешного `service.Update(...)` вставьте:

```
logger.Info($"UPDATE id={updated.Id} title=\"{updated.Title}
\");
```

Шаг 9. Логируем ChangeStatus

Найдите блок `input == "3"` (Изменить статус).

После успешного изменения вставьте:

```
logger.Info($"STATUS id={updated.Id} newStatus={updated.Statu
s}");
```

Шаг 10. Логируем Delete

В блоке `input == "4"` после успешного удаления вставьте:

```
logger.Info($"DELETE id={id}");
```

Шаг 11. Логируем Import/Export/Backup

Backup (9)

После успешного создания бэкапа:

```
logger.Info($"BACKUP created file=\"{backupPath}\"");
```

Export (10)

После успешного экспорта:

```
logger.Info($"EXPORT file=\"{exportFile}\"");
```

Import (11)

Перед импортом (после подтверждения):

```
logger.Info($"IMPORT start file=\"{importPath}\"");
```

После успешного импорта:

```
logger.Info($"IMPORT success count={importedTasks.Count}");
```

Если импорт отменили или была ошибка — можно логировать `Error`:

```
logger.Error("IMPORT failed: " + ex.Message);
```

Шаг 12. Проверка (F5) и поиск файла лога

1. Запустите программу (F5)
2. Добавьте задачу, поменяйте статус, удалите
3. Закройте программу (0)
4. Найдите папку `logs` (путь рядом с программой)
5. Откройте файл вида:

- `app_2025-12-29.log`

Там должны быть строки INFO: ADD/STATUS/DELETE...

Шаг 13. Коммит №2

Сообщение:

- `feat: log app actions to daily log file`

Commit All → Push.

4) Практика. Пара 3: команда “Показать лог” + отчёт

Добавим пункт меню, чтобы студент мог показать лог прямо из программы.

Шаг 14. Добавить пункт меню 14

В меню добавьте:

```
Console.WriteLine("14) Показать последние строки лога");
```

Шаг 15. Реализовать input == "14"

Добавьте блок:

```
if (input == "14")
{
    try
    {
        // Лог-файл за сегодня
        var logFile = Path.Combine(logsFolder, $"app_{DateTime.Now:yyyy-MM-dd}.log");

        if (!File.Exists(logFile))
        {
            Console.WriteLine("Лог за сегодня не найден.");
            continue;
        }

        var lines = File.ReadAllLines(logFile);

        Console.WriteLine("Последние 20 строк лога:");
        Console.WriteLine("-----");

        int start = Math.Max(0, lines.Length - 20);
        for (int i = start; i < lines.Length; i++)
            Console.WriteLine(lines[i]);
    }
}
```



```
        Console.WriteLine("-----");  
    }  
    catch (Exception ex)  
    {  
        Console.WriteLine("Ошибка чтения лога: " + ex.Message);  
    }  
  
    continue;  
}
```

Шаг 16. Проверка пункта 14

1. Запустите программу
2. Сделайте пару действий (Add/Status)
3. Нажмите 14
4. Должны появиться последние строки лога

Шаг 17. Отчёт `docs/LR10_Report.md`

Создайте `docs/LR10_Report.md`:

```
# Отчёт ЛР10 – TaskTracker
```

```
## Что сделано
```

- Добавлен AppLogger (лог в папку logs, файл на день)
- Логируются действия:
 - ADD, UPDATE, STATUS, DELETE
 - BACKUP, EXPORT, IMPORT
- Добавлен пункт меню 14) показать последние строки лога

```
## Где лежит лог
```

Папка logs рядом с программой.

Файл вида: app_YYYY-MM-DD.log

Как проверить

- 1) Запустить (F5)
- 2) Добавить задачу, изменить статус, удалить
- 3) Открыть `logs/app_*.log` и убедиться, что строки появились
- 4) В программе нажать 14 и увидеть последние строки

Шаг 18. Коммит №3

Сообщение:

- `feat: add log viewer command and LR10 report`

Commit All → Push.

5) Что сдавать (чек-лист)

✓ ссылка на репозиторий

✓ `docs/LR10_Report.md`

✓ видео/скрин:

- сделали действие (Add/Status)
- показали лог (14)
- показали файл `logs/app_...log`

✓ минимум 3 коммита

6) Оценивание

"3" — лог пишется в файл при Add/Status/Delete

"4" — лог пишется по всем ключевым действиям + пункт 14

"5" — аккуратные сообщения, логируются ошибки (`logger.Error`), понятные форматы строк